Lab 7:

For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.

```
// TopNDriver.java
package samples.topn;
public class TopNDriver {
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();

    Job wcJob = Job.getInstance(conf, "word count");
    wcJob.setJarByClass(TopNDriver.class);
    wcJob.setMapperClass(WordCountMapper.class);
    wcJob.setCombinerClass(WordCountReducer.class);
    wcJob.setReducerClass(WordCountReducer.class);
    wcJob.setOutputKeyClass(Text.class);
    wcJob.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(wcJob, new Path(args[0]));
    Path tempDir = new Path(args[1]);
    FileOutputFormat.setOutputPath(wcJob, tempDir);
    if (!wcJob.waitForCompletion(true)) System.exit(1);

    Job topJob = Job.getInstance(conf, "top 10 words");
    topJob.setJarByClass(TopNDriver.class);
    topJob.setMapperClass(TopNMapper.class);
    topJob.setReducerClass(TopNReducer.class);
    topJob.setMapOutputKeyClass(IntWritable.class);
    topJob.setMapOutputValueClass(Text.class);
    topJob.setOutputKeyClass(Text.class);
    topJob.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(topJob, tempDir);
    FileOutputFormat.setOutputPath(topJob, new Path(args[2]));
    System.exit(topJob.waitForCompletion(true) ? 0 : 1);
  }
}

// WordCountMapper.java
public class WordCountMapper extends Mapper<Object,Text,Text,IntWritable> {
  private static final IntWritable ONE = new IntWritable(1);
  private Text word = new Text();
  private String tokens = "[_|$#<>\^=\[\]\*/\\,;,.\-:()?!"']";
  protected void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
    String clean = value.toString().toLowerCase().replaceAll(tokens, " ");
    StringTokenizer itr = new StringTokenizer(clean);
    while (itr.hasMoreTokens()) {
      word.set(itr.nextToken().trim());
      context.write(word, ONE);
    }
  }
}

// WordCountReducer.java
public class WordCountReducer extends Reducer<Text,IntWritable,Text,IntWritable> {
```

```
  protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
    int sum = 0;
    for (IntWritable val : values) sum += val.get();
    context.write(key, new IntWritable(sum));
  }
}

// TopNMapper.java
public class TopNMapper extends Mapper<Object,Text,IntWritable,Text> {
  protected void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
    String[] parts = value.toString().split("\t");
    if (parts.length == 2)
      context.write(new IntWritable(Integer.parseInt(parts[1])),
                    new Text(parts[0]));
  }
}

// TopNReducer.java
public class TopNReducer extends Reducer<IntWritable,Text,Text,IntWritable> {
  private TreeMap<Integer,List<String>> countMap = new TreeMap<>(Collections.reverseOrder());
  protected void reduce(IntWritable key, Iterable<Text> values, Context context) {
    int cnt = key.get();
    List<String> words = countMap.getOrDefault(cnt, new ArrayList<>());
    for (Text w : values) words.add(w.toString());
    countMap.put(cnt, words);
  }
  protected void cleanup(Context context) throws IOException, InterruptedException {
    List<WordCount> topList = new ArrayList<>();
    int seen = 0;
    for (Map.Entry<Integer,List<String>> e : countMap.entrySet()) {
      for (String w : e.getValue()) {
        topList.add(new WordCount(w, e.getKey()));
        if (++seen == 10) break;
      }
      if (seen == 10) break;
    }
    Collections.sort(topList, (a, b) -> a.word.compareTo(b.word));
    for (WordCount wc : topList)
      context.write(new Text(wc.word), new IntWritable(wc.count));
  }
  private static class WordCount {
    String word; int count;
    WordCount(String w, int c) { word=w; count=c; }
  }
}

// Execution
$ hdfs dfs -mkdir /input_dir
$ hdfs dfs -copyFromLocal C:\input.txt /input_dir
$ hadoop jar C:\sort.jar samples.topn.TopN \
    /input_dir/input.txt /tmp/output_dir /output_dir
$ hdfs dfs -cat /output_dir/*
```

```
2026-05-03 19:54:54,582 INFO client.DefaultNoHAMRFailoverProxyProvider: Connecting to ResourceMana
2026-05-03 19:55:13,792 INFO mapreduce.Job: map 0% reduce 0%
2026-05-03 19:55:20,028 INFO mapreduce.Job: map 100% reduce 0%
2026-05-03 19:55:33,199 INFO mapreduce.Job: map 100% reduce 100%
2026-05-03 19:55:33,334 INFO mapreduce.Job: Job job_1620483374279_0001 completed successfully

// hdfs dfs -cat /output_dir/*:
bye    1
hadoop 1
hello  2
world  1
```